

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA51 COURSE NAME: Cryptography and Network Security

COURSE OUTCOME COVERED

CO3: *Examine cryptographic hash algorithms, digital signature schemes, and assess Kerberos and X.509 authentication frameworks for ensuring integrity, authentication, and non-repudiation.* CO (BL4 , BL5)

Assessment Tool Weightage: 20%

QUESTIONS: 5

TOTAL MARKS: 25

Annexure A CASE STUDY

Code of Conduct:

I Atchaya Chandran R (192521394) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

R. Atchaya

1. Weak Password Hashing in Web Application

PDF

- **Vulnerability Analysis:** Storing passwords using unsalted SHA-1 leaves systems vulnerable to rainbow table attacks. Without unique salts, identical passwords produce identical hashes, enabling attackers to batch-crack credentials. Furthermore, SHA-1 is a fast cryptographic hash function prone to collision vulnerabilities, allowing modern hardware to execute billions of brute-force attempts per second.

PDF+ 2

- **Recommended Strategy:** Transition to memory-hard Key Derivation Functions (KDFs) such as **Argon2id** or **bcrypt**, paired with a unique, cryptographically secure 128-bit salt per user.

PDF

- **Justification:** Argon2id and bcrypt introduce configurable work factors (cost parameters for CPU and memory usage). This deliberate delay makes GPU-accelerated parallel brute-force attacks computationally infeasible while neutralizing precomputed rainbow table attacks.

PDF+ 1

2. Digital Signature Compromise in Financial System

PDF

- **Impact on Security Principles:**
 - *Authentication:* Severely compromised, as the system can no longer distinguish between legitimate requests from the user and unauthorized actions performed by the attacker holding the private key.

PDF

- *Non-Repudiation:* Completely undermined. The legitimate owner can plausibly deny making fraudulent transactions, making audit logs legally non-binding.

PDF+ 1

- **Mitigation Strategies:**
 - **Secure Key Storage:** Enforce the storage of private keys in dedicated Hardware Security Modules (HSMs) or hardware-backed Trusted Platform Modules (TPMs) to prevent key extraction.

PDF

- **Multi-Factor Authorization:** Require Multi-Factor Authentication (MFA) and out-of-band transaction signing for high-value operations.

PDF

- **Revocation Protocols:** Implement real-time Online Certificate Status Protocol (OCSP) checking to immediately revoke compromised key certificates.

PDF

3. Kerberos Authentication Failure in Distributed System

PDF

- **Impact of Time Synchronization:** Kerberos uses timestamps inside encrypted tickets (e.g., Ticket-Granting Tickets) to prevent replay attacks. If the clock drift between a client, server, or Key Distribution Center (KDC) exceeds the default tolerance (typically 5 minutes), the receiver rejects the ticket as expired or fraudulent, causing systemic authentication failures.

PDF+ 1

- **Proposed Solutions:**

- **NTP Infrastructure:** Deploy a redundant Network Time Protocol (NTP) or Precision Time Protocol (PTP) synchronization architecture across all network domain controllers and hosts.

PDF

- **Automated Drift Monitoring:** Implement automated threshold alerts for time-skew anomalies to resolve clock synchronization issues prior to ticket failure.

PDF

4. Certificate Authority Breach in PKI System

PDF

- **Impact on System Trust:** A CA breach destroys the fundamental trust root of Public Key Infrastructure (PKI). Attackers holding compromised CA signing keys can issue fraudulent X.509 certificates for any target domain, enabling undetected Man-in-the-Middle (MitM) interception and traffic decryption.

PDF+ 1

- **Detection & Mitigation Mechanisms:**
 - **Certificate Transparency (CT):** Require mandatory CT logging so domain owners receive real-time alerts whenever a certificate is issued for their domain.

PDF

- **OCSP Stapling & Revocation:** Publish immediate Certificate Revocation Lists (CRLs) and force OCSP Stapling to revoke illegitimate certificates promptly.

PDF

- **Domain Protections:** Enforce CAA (Certification Authority Authorization) DNS records to restrict which CAs are authorized to issue certificates for specific domain spaces.

PDF

5. Integrity Verification in Cloud Data Storage

PDF

- **Failure Analysis:** Plain cryptographic hashing (e.g., SHA-256) verifies data integrity only if the hash itself is stored in a untamperable, out-of-band location. If an attacker modifies the cloud file and recalculates a new hash stored on the same server, simple hash verification checks will incorrectly pass.

PDF+ 1

- **Recommended Mechanism:** Implement **Keyed-Hash Message Authentication Codes (HMAC)** or **Authenticated Encryption with Associated Data (AEAD)** like AES-GCM, managed via client-side processing.

PDF

- **Justification:** HMAC combines a cryptographic hash with a secret key held exclusively by the user. An attacker who tampers with stored files cannot generate a valid HMAC without the secret key, guaranteeing end-to-end data integrity and origin authenticity regardless of cloud storage compromises.
- **1. OS Command Injection in System Utilities**
- **Vulnerability Analysis:** Occurs when an application passes unsanitized user input directly to a system shell execution context (e.g., `system()`, `exec()`). Attackers leverage command separators like `;`, `&&`, or `|` to execute arbitrary system commands, leading to full remote code execution (RCE) and host compromise.
- **Remediation Strategy & Justification:** Avoid invoking OS shell commands directly. Use built-in language APIs or execute commands via parametrized parameter arrays (e.g., `subprocess.run(["ping", "-c", "1", host])`) without invoking a shell interpreter (`shell=False`), ensuring input is strictly treated as positional arguments.
- **2. Server-Side Request Forgery (SSRF) in Cloud Infrastructure**

- **Vulnerability Analysis:** SSRF allows an attacker to force a server-side application to make HTTP/TCP requests to arbitrary internal or external resources. In cloud environments (e.g., AWS/GCP), attackers target local metadata endpoints (like http://169.254.169.254) to extract IAM credentials, gaining unauthorized access to infrastructure.
- **Remediation Strategy & Justification:** Enforce strict URL allowlisting, block access to private IPv4/IPv6 ranges (RFC 1918), and mandate IMDSv2 (Instance Metadata Service Version 2), which enforces session-oriented token headers that simple SSRF payloads cannot forge.
- **3. XML External Entity (XXE) Processing Vulnerabilities**
- **Vulnerability Analysis:** Legacy or misconfigured XML parsers process external entity declarations (<!ENTITY xxe SYSTEM "file:///etc/passwd">) by default. Unsanitized XML payloads allow attackers to exfiltrate local files, perform internal network SSRF, or trigger denial-of-service attacks during document parsing.
- **Remediation Strategy & Justification:** Explicitly disable external entity resolution (DTD processing) directly in the XML parser configuration (e.g., setting setFeature("http://xml.org/sax/features/external-general-entities", false)).
- **4. Broken Object Level Authorization (BOLA) in Microservices**
- **Vulnerability Analysis:** Microservices frequently rely on internal service-to-service communication without passing user authorization context down the call graph. An attacker tampering with resource identifiers at an ingress API gateway can manipulate backend microservices into retrieving or modifying records belonging to other tenants.
- **Remediation Strategy & Justification:** Pass cryptographically signed user context (e.g., JWTs or OAuth2 scopes) down to every downstream service. Validate resource ownership at the individual data-access layer rather than relying exclusively on edge security controls.
- **5. Insecure Deserialization in Enterprise Middleware**
- **Vulnerability Analysis:** Flaws arise when untrusted data is deserialized without validation, reconstructing arbitrary objects in memory. Attackers construct malicious byte streams (gadget chains) that hijack execution flow during object instantiation, resulting in unauthenticated remote code execution.
- **Remediation Strategy & Justification:** Avoid deserializing raw binary objects from untrusted sources. Migrate to safe structured data formats like JSON or Protocol Buffers, or enforce strict object type filtering (allowlisting) before instantiation.